Software Quality Assurance and Management

第6周：质量指标、分析与改进

Week 6: Quality Metrics, Analysis & Improvement

By: Fahad Sabah

计算机学院,
北京工业大学,中国
College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn

质量指标、分析与改进
Quality Metrics, Analysis & Improvement

- What should we measure?
- How do we analyze the measurements?
- How do we improve software quality based on the results?

Software quality metrics are **numerical indicators** used to understand whether software is **reliable, maintainable, secure, efficient, and useful** for users.

Similar to business quality KPIs, software teams use metrics to track quality, find weak points, and decide improvement actions.

- 我们应该衡量什么？
- 我们如何分析这些测量数据？
- 我们如何根据结果提升软件质量？

软件质量指标是用来判断软件是否可靠、可维护、安全、高效且对用户有用。

类似于业务质量关键绩效指标（KPI），软件团队利用指标来跟踪质量、发现薄弱环节并决定改进措施。

简单定义
Simple Definition

What are Software Quality Metrics?

- Software quality metrics are measurable values used to evaluate the **quality of software products, development processes, and user satisfaction**.

In simple words:

Software quality metrics tell us whether the software is good, where it is weak, and how we can improve it.

什么是软件质量指标?

- 软件质量指标是用于评估软件产品质量、开发流程和用户满意度的可衡量数值。

简单来说：

软件质量指标告诉我们软件是否优秀，存在哪些弱点，以及如何改进它。

质量指标、分析与改进
Quality Metrics, Analysis & Improvement

Examples:

Question	Metric
Does the software have many bugs?	Defect density
Are bugs fixed quickly?	Defect resolution time
Is the software secure?	Number of vulnerabilities
Are users happy?	Customer satisfaction score
Is the team delivering on time?	Sprint velocity, lead time

为什么软件质量指标重要?
Why Software Quality Metrics Are Important?

Software quality is difficult to judge only by opinion. Metrics help us make decisions based on data.

软件质量很难仅凭个人观点来评判。指标帮助我们基于数据做出决策。

They help teams:

- ☐ Find bugs earlier.
- ☐ Improve software reliability.
- ☐ Reduce maintenance cost.
- ☐ Improve customer satisfaction.
- ☐ Improve team productivity.
- ☐ Track whether quality is getting better or worse.

他们帮助球队:

- ❑ 更早发现虫子。
- ❑ 提升软件可靠性。
- ❑ 降低维护成本。
- ❑ 提升客户满意度。
- ❑ 提升团队生产力。
- ❑ 跟踪质量是变好还是变差。

软件质量指标的主要类别
Main Categories of Software Quality Metrics

1. Agile Metrics
2. Production Metrics
3. Security Responses Metrics
4. Dependencies Age
5. Size-Oriented Measurements
6. Function-Oriented Methods
7. Defect Metrics
8. Pull Request
9. QA Metrics
10. Customer Satisfaction

敏捷指标
Agile Metrics

Agile metrics measure how well the software development team is working.

Metric	Meaning
Sprint velocity	How much work is completed in one sprint
Burndown chart	Whether the team is finishing tasks on time
Lead time	Time from requirement to delivery
Cycle time	Time to complete one task

敏捷指标
Agile Metrics

Example

A team completes:

Sprint	**Story Points Completed**
Sprint 1	25
Sprint 2	30
Sprint 3	28

Average velocity: $\frac{25+30+28}{3} = 27.67$

So, the team can plan about 28 story points for the next sprint.

生产指标
Production Metrics

Production metrics measure the quality of software after deployment.

Metric	Meaning
System uptime	How long the system works without failure
Error rate	Percentage of failed requests
Response time	How fast the system responds
Incident frequency	Number of production problems

Example

If a website receives 10,000 requests and 200 fail:

$$Error\ Rate = 200/10000 \times 100 = 2\%$$

A lower error rate means better production quality.

安全响应指标
Security Response Metrics

Security metrics measure how safe the software is.

Metric	Meaning
Number of vulnerabilities	Security problems found
Mean time to detect	Time needed to find a security issue
Mean time to fix	Time needed to fix a security issue
Patch delay	Time between patch release and installation

Simple Analysis

If vulnerabilities are increasing every month, the team may need:

- ❑ More secure coding practices.
- ❑ Security testing before release.
- ❑ Dependency updates.
- ❑ Developer security training

抚养年龄
Dependency Age

Modern software uses many external libraries. Old dependencies may create security and compatibility problems.

Metric	Meaning
Number of outdated packages	How many libraries are old
Average dependency age	Average age of used libraries
Critical vulnerable dependencies	Dangerous outdated libraries

Example

A project has 20 dependencies.

8 are outdated.

$$Outdated\ Dependency\ Rate = 8/20 \times 100 = 40\%$$

This means dependency maintenance is weak.

以规模为导向的度量
Size-Oriented Metrics

Size-oriented metrics measure software based on its size.

Metric	Meaning
Lines of Code, LOC	Number of code lines
Defects per KLOC	Bugs per 1000 lines of code
Productivity	LOC written per developer per month

Example

A project has 50 bugs and 10,000 lines of code.

$$Defect\ Density = 50/10 = 5\ defects/KLOC$$

This means there are 5 bugs per 1000 lines of code.

面向功能的度量
Function-Oriented Metrics

Function-oriented metrics measure software based on features or functions, not code size.

Metric	Meaning
Function points	Functional size of software
Inputs	Data entered into the system
Outputs	Reports or results generated
User interactions	User operations supported

Explanation

A small program may have many important functions, while a large program may have many repeated code lines. Therefore, function-oriented metrics are useful when we want to measure software based on user functionality.

质量指标、分析与改进
Defect Metrics

Defect metrics are among the most important software quality metrics.

Metric	Meaning
Defect density	Number of defects per software size
Defect severity	How serious the bugs are
Defect leakage	Bugs found by users after release
Reopen rate	Fixed bugs that appear again
Defects fixed per day	How many bugs are fixed daily

$$Defect\ Leakage\ (\%) = \frac{\text{Defects found after release}}{Total\ defects\ found\ (before\ +\ after\ release)} \times 100$$

Example

A software team found 100 bugs before release.

After release, users found 20 more bugs.

$$Defect\ Leakage = 20/(100 + 20)\ \times 100 = 16.67\%$$

A high defect leakage means testing before release was not enough.

拉取请求指标
Pull Request Metrics

Pull request metrics measure the quality of code review.

Metric	Meaning
PR review time	Time needed to review code
PR rejection rate	Percentage of PRs needing major changes
Number of comments	Review discussion level
Merge frequency	How often code is merged

Interpretation

- ❑ If pull requests take too long to review, delivery becomes slow.
- ❑ If pull requests are merged too quickly without review, bugs may increase.
- ❑ A good balance is needed.

QA指标
QA Metrics

QA metrics measure the effectiveness of the testing process.

Metric	Meaning
Test coverage	Percentage of code tested
Test pass rate	Percentage of tests passed
Automation rate	Percentage of tests automated
Test execution time	Time needed to run tests
Escaped bugs	Bugs missed by QA

Example

A project has 500 test cases.

450 passed.

$$Test\ Pass\ Rate = 450/500 \times 100 = 90\%$$

This means the current build quality is acceptable, but the failed 10% must be analyzed.

客户满意度指标
Customer Satisfaction Metrics

Customer satisfaction metrics measure whether users are happy with the software.

Metric	Meaning
NPS	User recommendation score
Customer complaints	Number of complaints
Average rating score	App or product rating
Retention rate	Percentage of users who continue using the software
Average response time	Time to respond to customer issues

Net Promoter Score, number of complaints, customer retention rate, average time to solve, average time to respond, and average rating score as important quality KPIs.

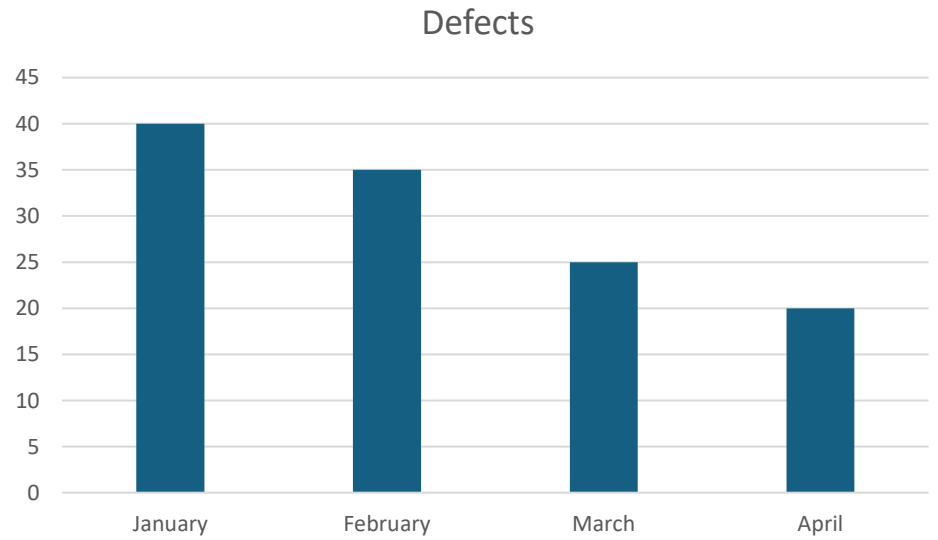
软件质量分析
Software Quality Analysis

Metrics are only useful if we analyze them correctly.

Trend Analysis

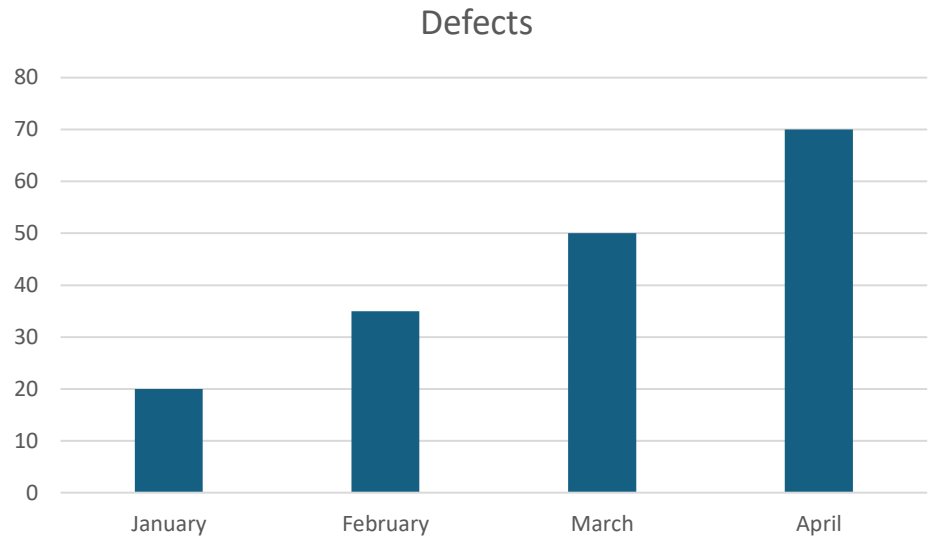
❑ Trend analysis means comparing the same metric over time.

Example:



This shows improvement because defects are decreasing.

But if:



This shows quality is getting worse.

比较分析
Comparison Analysis

Compare one team, project, or release with another.

Example:

Release	**Defect Density**
Version 1.0	6 defects/KLOC
Version 2.0	3 defects/KLOC
Version 3.0	2 defects/KLOC

Version 2.0 has better quality.
Version 3.0 is even better.

根本原因分析
Root Cause Analysis

When a metric is bad, we should ask why.
Example:
Problem: Defect leakage is high.

Possible causes:

1. Test cases are incomplete.
2. Requirements are unclear.
3. Developers do not write unit tests.
4. Code review is weak.
5. Testing time is too short.

A useful method is the 5 Whys method.
Example:
Problem: Many bugs are found after release.

1. **Why?** Because QA missed them.
2. **Why did QA miss them?** Because test cases did not cover edge cases.
3. **Why?** Because requirements were unclear.
4. **Why?** Because product and development teams did not discuss details.
5. **Why?** Because there was no requirement review meeting.

Improvement: Add requirement review before development.

Pareto Analysis

Vilfredo Pareto, an Italian economist, who in 1896 observed that roughly 80% of the land in Italy was owned by 20% of the population.

Pareto analysis means focusing on the biggest causes first.

Example:

Defect Cause	Number of Bugs
Requirement misunderstanding	40
Coding error	25
UI issue	15
Database issue	10
Deployment issue	5

≈80% of the effects come from ≈20% of the causes.

The biggest problem is requirement misunderstanding.
So, the first improvement should be better requirement analysis.

Pareto Analysis

The method involves:

1. Listing the problems or causes.
2. Quantifying their frequency or impact (e.g., number of defects, cost, time).
3. Sorting them in descending order of importance.
4. Drawing a Pareto chart (a bar graph with a cumulative percentage line) to highlight the “vital few” items that contribute to the majority of the issue.

The goal is to prioritize efforts where they will make the biggest difference.

Pareto Analysis

Imagine a software team logs these post-release bugs:

Bug Category	Number of Bugs
Login issues	95
Checkout failures	70
UI misalignment	20
Performance lag	10
Other	5
Total	200

Pareto Analysis

Step 1: Sort by frequency (already sorted above).
Step 2: Calculate percentages and cumulative percentage.

Category	Bugs	% of Total	Cumulative %
Login	95	47.5%	47.5%
Checkout	70	35.0%	82.5%
UI	20	10.0%	92.5%
Perf	10	5.0%	97.5%
Other	5	2.5%	100.0%

Pareto insight: The first two categories (Login + Checkout) represent only 40% of the bug types (2 out of 5), but they cause 82.5% of the bugs.

软件质量改进
Software Quality Improvement

After analysis, we need action.

Improve Requirements

Poor requirements cause many defects.

Improvement actions:

- ☐ Write clear user stories.
- ☐ Add acceptance criteria.
- ☐ Review requirements before coding.
- ☐ Use examples and diagrams.

提升代码质量
Improve Code Quality

Code quality affects maintainability and reliability.

Improvement actions:

- ❑ Use coding standards.
- ❑ Use static code analysis.
- ❑ Reduce code complexity.
- ❑ Remove duplicate code.
- ❑ Refactor difficult modules.

Important metrics:

Metric	Improvement Target
Code complexity	Lower
Duplicate code	Lower
Code smells	Lower
Maintainability index	Higher

改进测试
Improve Testing

Testing directly reduces defects.

Improvement actions:

- ❑ Increase unit test coverage.
- ❑ Add integration testing.
- ❑ Add regression testing.
- ❑ Automate repeated tests.
- ❑ Test high-risk modules first.

Important metrics:

Metric	Target
Test coverage	Higher
Test pass rate	Higher
Escaped defects	Lower
Test automation rate	Higher

提升安全性
Improve Security

Security quality should be measured continuously.

Improvement actions:

- ❑ Update dependencies.
- ❑ Use vulnerability scanning.
- ❑ Review authentication and authorization.
- ❑ Fix high-severity vulnerabilities first.
- ❑ Train developers in secure coding.

Important metrics:

Metric	Target
Critical vulnerabilities	0
Patch delay	Lower
Dependency age	Lower
Security response time	Lower

提升客户满意度
Improve Customer Satisfaction

Software quality is not only technical. Users must also feel satisfied.

Improvement actions:

- ❑ Reduce response time.
- ❑ Solve complaints quickly.
- ❑ Monitor user feedback.
- ❑ Improve usability.
- ❑ Fix frequent customer problems first.

Important metrics:

Metric	Target
NPS	Higher
Complaints	Lower
Average rating	Higher
Time to solve	Lower

案例研究
Case Study

Case: Online Shopping App

A company releases a shopping app. After release, the following data is collected:

Metric	Value
Defects found before release	80
Defects found after release	40
Test coverage	55%
Average response time	4 seconds
Customer complaints per month	120
Outdated dependencies	35%
Average bug fixing time	6 days

分析
Analysis

The software has several quality problems:

- High defect leakage
  40/(80+40)×100=33.3%
- Low test coverage
  Only 55%, so many code parts are not tested.
- Slow response time
  4 seconds may make users unhappy.
- Many customer complaints
  120 complaints per month shows poor user experience.
- Outdated dependencies
  35% outdated dependencies may create security risks.

该软件存在若干质量问题：

- 高缺陷泄漏
  40/(80+40)×100=33.3%
- 低测试覆盖率
  只有55%，所以很多代码部分没有被测试。
- 响应时间较慢
  4秒钟可能会让用户不满意。
- 许多客户投诉
  每月120起投诉显示用户体验很差。
- 过时的依赖
  35%的过时依赖可能带来安全风险。

改进计划
Improvement Plan

	Problem	Improvement
1.	High defect leakage	Add regression testing and better QA review
2.	Low test coverage	Increase unit and integration tests
3.	Slow response time	Optimize database queries and caching
4.	Many complaints	Analyze complaint categories and fix top issues
5.	Old dependencies	Schedule monthly dependency updates
6.	Slow bug fixing	Prioritize severe bugs and improve debugging process

活动
Activity

A student management system has:

Metric Value

1.60 bugs before release
2.30 bugs after release
3.70% test pass rate
4.5 days average bug fix time
5.25% outdated dependencies
6.3.2/5 user rating

Answer:

Which metric is the most serious?
What does the metric show?
What is the possible cause?
What improvement action should be taken?

30 bugs found after release — because it directly represents escaped defects reaching real users, which damages trust, increases support cost, and may disrupt learning.

Why not the others?

Defect Leakage = 30 / (60 + 30) = 30 / 90 = 33.3%

Low test effectiveness (70% pass rate)

Slow bug fixing (5 days average)

Outdated dependencies (25%)

质量指标、分析与改进
Quality Metrics, Analysis & Improvement

1. Defect Tracking & Escaped Defect Management

- ❑ Jira (Atlassian) – https://www.atlassian.com/software/jira
- ❑ Azure DevOps Boards – https://azure.microsoft.com/en-us/products/devops
- ❑ GitLab Issues – https://about.gitlab.com/
- ❑ Redmine – https://www.redmine.org/

2. Test Coverage Analysis

- ❑ SonarQube – https://www.sonarsource.com/products/sonarqube/
- ❑ Istanbul (nyc) – https://istanbul.js.org/
- ❑ JaCoCo – https://www.jacoco.org/jacoco/
- ❑ Coverage.py – https://coverage.readthedocs.io/
- ❑ dotCover (JetBrains) – https://www.jetbrains.com/dotcover/
- ❑ SimpleCov – https://github.com/simplecov-ruby/simplecov

质量指标、分析与改进
Quality Metrics, Analysis & Improvement

3. Operational Metrics & MTTR (Incident Management, Monitoring)

- ❑ PagerDuty – https://www.pagerduty.com/
- ❑ Opsgenie (Atlassian) – https://www.atlassian.com/software/opsgenie
- ❑ ServiceNow – https://www.servicenow.com/
- ❑ Jira Service Management – https://www.atlassian.com/software/jira/service-management
- ❑ Splunk On-Call (VictorOps) – https://www.splunk.com/en_us/software/splunk-on-call.html
- ❑ Datadog – https://www.datadoghq.com/
- ❑ New Relic – https://newrelic.com/

4. Root Cause Analysis (RCA) & Quality Analysis Techniques

- ❑ Lucidchart – https://www.lucidchart.com/
- ❑ Miro – https://miro.com/
- ❑ MindMeister – https://www.mindmeister.com/
- ❑ Sologic Root Cause Analysis – https://www.sologic.com/
- ❑ Apollo RCA (ARMS Reliability) – https://www.apollorootcause.com/

质量指标、分析与改进
Quality Metrics, Analysis & Improvement

5. Quality Dashboards (Power BI, Grafana & Alternatives)

- ❑ Grafana – https://grafana.com/
- ❑ Power BI – https://powerbi.microsoft.com/
- ❑ Kibana – https://www.elastic.co/kibana/
- ❑ Tableau – https://www.tableau.com/
- ❑ Metabase – https://www.metabase.com/